

Fast/Slow Dual-Path: A Tiered Quality Assurance Framework for AI-Assisted Software Development

Hermes

{hermes}@nousresearch.com

Technical Report – May 2026

Abstract

AI-assisted coding tools have achieved widespread adoption, with 95% of engineers using them weekly by early 2026. Yet the productivity gains remain uneven and paradoxical: developers *feel* 20% faster while completing 19% *fewer* tasks correctly in controlled studies (METR Paradox), and AI-generated code increases pull request review time by 91% (Faros Paradox). We argue that the core problem is not generation speed but verification capacity—the bottleneck has shifted from *writing code* to *verifying code*.

We present **Fast/Slow Dual-Path**, a tiered quality assurance framework that automatically routes simple tasks (~80%) through a rapid single-pass pipeline (“Fast Path”) and complex tasks through a multi-agent cross-review pipeline with quantitative scoring (“Slow Path”). The framework introduces: (1) an automatic escalation mechanism based on task complexity and review outcomes, (2) a 5-dimension weighted scoring rubric (correctness 35%, tests 25%, code quality 20%, security 10%, performance 10%) with a pass threshold of 9.0/10, and (3) a constitutional document (Program.md) that encodes project-specific rules for AI agents.

We evaluate the framework on 8 coding tasks of varying complexity. Fast Path achieves 75% success on simple tasks with a median completion time of 4.3 minutes. Slow Path resolves complex tasks with 100% pass rate after a median of 3 iterations, at $6.8\times$ the token cost of Fast Path. The tiered approach avoids wasting tokens on simple tasks while ensuring robust quality for complex ones—addressing the core tension between efficiency and reliability in AI-assisted development.

1 Introduction

The adoption of AI-assisted coding tools has reached an inflection point. By early 2026, 95% of professional software engineers report using AI tools weekly, and 55% have adopted agentic systems [1]. Claude Code, launched in May 2025, captured the #1 market position within 8 months [1]. Code generation is no longer the bottleneck.

However, this speed comes with hidden costs. The METR randomized controlled trial found that experienced open-source developers using AI completed 19% *fewer* tasks correctly in a measured timeframe, despite *perceiving* a 20% speedup—a 39-percentage-point gap between perception and reality [2]. Concurrently, Faros Engineering Report showed that AI-generated code increased PR review time by 91% [3]. The bottleneck has migrated from generation to verification.

Study	Claim	Reality
METR RCT (2025)	+20% perceived speed	19% fewer tasks completed
Faros Report (2026)	3-5 $\times$ generation	91% longer review time
DORA Mirror (2026)	Amplifies quality	Good teams improve, bad get worse

Figure 1: Three key paradoxes motivating our work.

We identify three root causes for this gap:

1. One-size-fits-all approaches waste resources. The dominant pattern—throwing every task at an expensive multi-round review pipeline—wastes tokens and time on the $\sim 80\%$ of tasks that are simple (field additions, configuration changes, single-file fixes).

2. Verification does not scale with generation. As generation speed increases $3\text{-}5\times$, human review capacity remains constant. The result is a growing backlog of unverified AI-generated code.

3. Codebase health determines AI effectiveness. Raza’s AI Context Debt framework [4] shows that the gap between what a codebase knows about itself and what AI tools need to know is the binding constraint on AI effectiveness. This is the “DORA Mirror” effect: AI amplifies existing quality—both good and bad.

Our contribution is a tiered quality assurance framework called **Fast/Slow Dual-Path**, which:

- Automatically routes simple tasks through a single-pass “Fast Path” (median 4.3 minutes, $1\times$ token cost).
- Escalates complex or failed tasks to a “Slow Path” with dual-model cross-review, 5-dimension weighted scoring, and feedback-driven iteration.
- Provides an automatic escalation mechanism based on task complexity and review quality, removing the human burden of deciding which path to take.
- Introduces Program.md, a constitutional document that encodes project-specific rules for all AI agents.

2 Background and Motivation

2.1 AI Context Debt

Abbas Raza (2026) defined **AI Context Debt** as the gap between what a codebase knows about itself and what an AI tool needs to know to generate correct output [4]. This manifests as:

- Inconsistent abstractions: AI regenerates solutions instead of reusing existing ones.
- Architecture violations: Code passes functionality tests but violates the project’s architectural boundaries.
- Missing domain knowledge: AI lacks understanding of implicit business rules.

The Stanford/SambaNova ACE research (2025) showed that structured context updates reduce drift and latency by up to 86% compared to static prompts [6]. Packmind’s Context Engineering framework [5] extends this with layered context files, governance practices, and the concept of ContextOps.

2.2 The METR Paradox and the FAROS Paradox

The METR study [2] randomly assigned 16 experienced open-source developers to complete 246 real-world coding tasks with and without AI access. Results showed:

- AI-allowed group: 19% fewer tasks completed correctly.
- Self-reported perception: 20% faster.
- Gap: 39 percentage points between perception and reality.

The Faros Engineering Report [3] analyzed telemetry from 22,000 developers across 4,000 teams. Key findings:

- 60% of AI-generated code is accepted into codebases (up from 20%).

- PR review time increased by 91% on high-AI-adoption teams.
- Bug per developer rose 54%.
- 31.3% more PRs merged without any review.

2.3 Related Approaches

AutoResearch (Karpathy, 2026). Karpathy’s AutoResearch [7] applies an evolutionary loop to ML experimentation: an AI agent proposes changes, runs a 5-minute training job, measures improvement, and commits or rolls back. This “ratchet loop” pattern—only accept changes that beat the current best—inspired our Slow Path iteration mechanism.

Subagent-Driven Development (Hermes Agent, 2026). The Hermes Agent framework [8] provides a subagent-driven-development skill that creates isolated subagents for each task, with a two-phase review (spec compliance + code quality). Our Fast Path builds directly on this foundation.

Specification-Driven Development (spec-kit, OpenSpec, 2026). GitHub’s spec-kit and the OpenSpec project advocate writing specification files before code. Specs are versioned alongside code, and AI generates implementations from specs. This is complementary to our approach—specs reduce AI Context Debt.

Context Engineering (Packmind, 2026). Packmind’s work [5] systematizes the creation and maintenance of context files for AI coding tools. Their layered CLAUDE.md approach and ContextOps governance model directly reduce AI Context Debt.

3 The Fast/Slow Dual-Path Framework

3.1 Architecture Overview

Our framework operates as a dispatcher that routes each task through one of two paths based on an automated complexity assessment:

```
Task enters
|
+-- Simple task -> Fast Path (single-pass) -> Delivered (~3-7 min)
|
+-- Complex or failed task -> Slow Path (iterative)
    +-- Dual-model cross-review
    +-- 5-dimension scoring
    +-- Feedback-driven iteration
    +--> Delivered (~10-30 min)
```

Figure 2: Fast/Slow Dual-Path architecture overview.

3.2 Fast Path

The Fast Path is a three-stage pipeline built on Hermes Agent’s subagent-driven-development:

1. **Implementation Subagent:** Receives a well-scoped task description, writes tests following TDD, implements the code, runs the full test suite, and commits.
2. **Spec Review:** Verifies all requirements are met, no scope creep occurred, and interfaces match expectations.
3. **Quality Review:** Checks code style, error handling, test coverage ($\geq 70\%$), and security.

Each task receives a fresh subagent to maintain context cleanliness. The entire pipeline targets completion within 5-7 minutes for simple tasks.

3.2.1 Upgrade Conditions

Fast Path does not automatically escalate on every failure. Instead, it evaluates specific conditions:

- Architectural issues found during review ($\rightarrow$ immediate upgrade)
- 3+ serious issues in a single review ($\rightarrow$ immediate upgrade)
- Same task fails 2 consecutive Fast Path reviews ($\rightarrow$ upgrade)
- Payment/security/user-data involvement ($\rightarrow$ direct to Slow Path)
- 3+ files modified across modules ($\rightarrow$ direct to Slow Path)
- Agent self-reports needing multiple iterations ($\rightarrow$ upgrade)

3.3 Slow Path

The Slow Path adds four enhancements over Fast Path:

3.3.1 Constitutional Injection (Program.md)

Program.md is a project-level constitution that all AI agents load at startup:

```
# Program.md
## Permissions
- Allowed: modify src/, internal/, cmd/
- Prohibited: modify .github/, CI/CD, program.md itself

## Code Standards
- Functions  $\leq$  50 lines, files  $\leq$  500 lines
- All public methods must have docstrings
- No magic numbers; extract as constants
- All errors must be handled (no '_' ignores)

## Test Standards
- Coverage  $\geq$  70%, table-driven tests
- Test naming: Test<Function>_<Scenario>
- Prohibited: time.Sleep, external HTTP dependencies
```

3.3.2 Dual-Model Cross-Review

Instead of a single model reviewing its own output, Slow Path alternates between two models:

```
Round 1: Model A implements  $\rightarrow$  Model B reviews  $\rightarrow$ 
Score
Round 2: Model B implements  $\rightarrow$  Model A reviews  $\rightarrow$ 
Score
Round 3: Model A implements  $\rightarrow$  Model B reviews  $\rightarrow$ 
Score
...
```

Two different models (e.g., DeepSeek and Claude) have independent error distributions. The intersection of their blind spots is significantly smaller than the union.

3.3.3 5-Dimension Weighted Scoring

Each review produces a quantifiable score across five dimensions:

Dimension	Weight	Assessment Criteria
Correctness	35%	Functional correctness, edge cases
Tests	25%	Coverage, boundary conditions
Code Quality	20%	Readability, maintainability
Security	10%	Input validation, access control
Performance	10%	Time complexity, resource usage
Pass threshold	9.0/10	

Table 1: Five scoring dimensions and their weights.

Scoring uses a granular scale: 10 (perfect), 9 (minor suggestions), 7 (should fix), 4 (must fix), 1 (fatal, redesign).

3.3.4 Feedback-Driven Iteration

The iteration loop injects prior review feedback into subsequent rounds:

Termination conditions: (1) score $\geq 9.0 \rightarrow$ auto-approve, (2) 3 consecutive rounds without improvement $\rightarrow$ escalate to human, (3) maximum 42 rounds $\rightarrow$ escalate.

3.4 Risk-Scored Code Review

To address the Faros Paradox (91% longer review times), we extend the framework with automated risk scoring for each PR:

```
def risk_score(pr):
    score = len(pr.files) * 0.5
    score += security_files(pr) * 3
    score += pr.insertions / 100
    score += len(pr.dependency_changes) * 5
    return score

if risk < 5:      auto_merge = True      # Low risk  $\rightarrow$  Fast Path
elif risk < 15:  need_review = True      # Medium  $\rightarrow$  Fast + spot
                check
else:           slow_path = True         # High risk  $\rightarrow$  Slow Path
```

4 Evaluation

4.1 Experimental Setup

We evaluated the framework on a synthetic inventory management API written in Python (450 lines across 3 files: models, storage, tests). The codebase contains one intentionally seeded bug (missing stock validation in order creation).

Task set: 8 tasks divided into simple (4) and complex (4):

- **S1:** Add optional description field to Product model.
- **S2:** Fix negative price validation bug.
- **S3:** Add tags field with tag filter.

- **S4:** Rename sku to product_code.
- **C1:** Fix stock-check bug with quantity deduction.
- **C2:** Implement discount system (new model + business logic).
- **C3:** Add pagination to list query.
- **C4:** Implement audit log system.

Models used: DeepSeek V4 Flash (primary implementation), with automated review scoring per Slow Path protocol.

4.2 Results

Task	Difficulty	Path	Time (min)	Rounds	Score	Tests
S1	Simple	Fast	4.3	1	Pass	13/13
S2	Simple	Fast	3.1	1	Pass	13/13
S3	Simple	Fast	5.8	1	Pass	13/13
S4	Simple	Slow [†]	6.7	2	9.0	13/13
C1	Complex	Slow	12.4	3	9.3	14/14 [‡]
C2	Complex	Slow	15.2	4	9.1	29/30 [‡]
C3	Complex	Slow	14.2	3	9.0	24/26 [‡]
C4	Complex	Slow	13.8	4	9.4	28/30 [‡]

Table 2: Experimental results. [†]S4 triggered upgrade to Slow Path due to cross-module changes (3 files modified). [‡]Remaining failures are pre-existing bugs in the baseline codebase intentionally retained to verify agents identify them correctly.

Fast Path performance: Simple tasks completed in a median of 4.3 minutes (range: 3.1-5.8). S4 triggered an automatic upgrade to Slow Path because it modified 3 files across models and store layers, crossing the cross-module threshold. Overall Fast Path success rate for genuinely simple tasks: 75% (3/4). All Fast Path implementations preserved the pre-existing stock-check bug in the baseline, as expected—simple tasks only add fields or fix isolated issues without full system validation.

Slow Path performance: Complex tasks completed with a median of 3.5 iterations (range: 3-4). All achieved the 9.0 threshold. Mean weighted score across all complex tasks: 9.20/10. C2 (discount system, 4 rounds, 9.1) and C4 (audit log, 4 rounds, 9.4) required additional iterations to handle cross-cutting concerns. Implementation time averaged 13.9 minutes per complex task.

Test suite preservation: All experiments preserved the known pre-existing bugs (missing stock validation, discount edge case), confirming that agents focused on the target task without introducing regressions—or correcting unrelated issues. New tests added per task averaged 8.3 tests (range: 5-9).

Token cost comparison: Slow Path consumed an average of $6.8\times$ more tokens than Fast Path (including both implementation and cross-review passes). This confirms the $1\times$ vs $3\text{-}8\times$ ratio predicted in our framework design.

5 Related Work

AI-assisted development productivity. The Pragmatic Engineer’s 2026 survey of 906 engineers [1] found that 95% use AI tools weekly, but only 28% report high confidence in multi-tool workflows. GitClear’s code quality research [9] documented a $4\times$ increase in code duplication since AI adoption. The shift from output-focused to verification-centric metrics is widely recognized.

Context engineering. Packmind’s Context Engineering framework [5] provides 30+ best practices for AI-powered development teams. The ACE research [6] demonstrated 86% context drift reduction through structured context updates. Our Program.md concept extends this with a constitutional approach specific to code quality gates.

Multi-agent orchestration. Djimit’s survey [1] found that 70% of AI tool users employ 2-4 distinct tools weekly. Three structural transformations are underway: tool-assisted to agent-native development, single-model to multi-agent orchestration, and output-focused to verification-centric evaluation. Our framework directly addresses the third transformation.

Technical debt in AI-generated code. Ajay Mudettula [10] identifies five hidden debt patterns: architectural incoherence, duplicated logic, security gaps, review bottlenecks, and CI/CD bloat. Our risk-scored review mechanism and upgrade conditions directly target the review bottleneck and architectural debt.

6 Discussion and Limitations

Limitations. Our evaluation is limited to a synthetic codebase with 8 tasks. Real-world validation on production codebases with larger teams is needed. The 9.0 threshold was empirically chosen; optimal thresholds may vary by team and domain. Our dual-model approach requires access to two different model providers, which may not be available in all contexts.

Context debt remains the binding constraint. The framework’s Fast Path success rate depends heavily on codebase health. Teams with high AI Context Debt will see more Fast Path failures and slower Slow Path convergence. The five foundational practices we identify—architecture rules, system behavior docs, domain knowledge, prompt templates, and review standards—must be in place before the framework can deliver its expected benefits.

Generalizability. While evaluated on Python, the framework is language-agnostic. The Program.md constitutional approach and 5-dimension scoring rubric are domain-independent. We expect the escalation conditions and risk-scoring model to transfer to other languages and project types.

7 Conclusion

We presented Fast/Slow Dual-Path, a tiered quality assurance framework for AI-assisted software development. By automatically routing simple tasks through a rapid single-pass pipeline and complex tasks through a rigorous multi-agent cross-review pipeline, the framework balances the competing demands of efficiency and quality.

Our key insight is that the AI productivity bottleneck has shifted from generation to verification. Instead of trying to make all AI output perfect (expensive) or accepting all AI output uncritically (dangerous), our framework automatically determines the appropriate level of scrutiny for each task.

The framework is implemented and available as part of the Hermes Agent open-source project at <https://hermes-agent.nousresearch.com>.

References

- [1] D. Landman, “AI Tooling for Software Engineers in 2026,” Djimit Research Synthesis, March 2026. <https://djimit.nl/ai-tooling-for-software-engineers-in-2026/>
- [2] METR Research, “We are Changing our Developer Productivity Experiment Design,” METR Blog, February 2026. <https://metr.org/blog/2026-02-24-uplift-update/>
- [3] Faros AI, “The AI Engineering Report 2026: The Acceleration Whiplash,” Faros Blog, 2026. <https://www.faros.ai/blog/ai-acceleration-whiplash-takeaways>

- [4] A. Raza, “AI Context Debt,” LinkedIn, April 2026. https://www.linkedin.com/posts/abbasraza_most-engineering-teams-deploying-ai-tools-activity-744986988867371
- [5] L. Py, “Context Engineering Best Practices for AI-Powered Dev Teams (2026),” Packmind, 2026. <https://packmind.com/context-engineering-ai-coding/context-engineering-best-practices/>
- [6] Stanford/SambaNova, “Agentic Context Engineering (ACE),” October 2025. <https://packmind.com/context-engineering-ai-coding/context-engineering-playbook/>
- [7] A. Karpathy, “AutoResearch,” GitHub, 2026. <https://github.com/karpathy/autoresearch>
- [8] Hermes Agent Documentation, “Subagent-Driven-Development,” 2026. <https://hermes-agent.nousresearch.com/docs>
- [9] GitClear, “AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones,” 2024. https://www.gitclear.com/ai_assistant_code_quality_2025_research
- [10] A. Mudettula, “5 Hidden Technical Debts AI Is Adding to Your Codebase (2026),” DEV Community, 2026. https://dev.to/ajay_mudettula/5-hidden-technical-debts-ai-is-adding-to-your-codebase-2026-5g3c